

Product Specification

NestUp

The problem, the users, and what the product is for

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product nestup-kappa.vercel.app

Repository github.com/nestupweb/nestup

1. The problem

Finding a room in a shared apartment is not really a search problem. The listings are easy to find — they are on Facebook groups, Yad2 and WhatsApp. What is hard is that **a room is not the thing you are choosing. The people are.**

The existing tools get this backwards. They index apartments: rent, rooms, square metres, neighbourhood. Everything that actually decides whether a shared flat works — when people sleep, how often guests come over, whether the kitchen is kept clean, whether someone smokes indoors, whether the flat keeps Shabbat — is either missing or buried in free text that nobody can filter on.

The result is a process that wastes everyone's time on both sides:

- **The seeker** messages thirty listings, gets eight replies, visits four apartments, and discovers the incompatibility in person — after travelling across the city.
- **The household** fields dozens of near-identical "hi, is it still available?" messages, repeats the same screening questions to each one, and still ends up meeting people who were never going to fit.

Both sides are doing manual filtering that a computer could do for them, and both sides are doing it *after* the expensive step (travelling, meeting) rather than before it.

There is a second, quieter problem. Once a conversation does start, it usually starts with **one** roommate — normally whoever posted the ad — while the decision belongs to the whole household. The other roommates hear a summary. NestUp treats the household, not the poster, as the counterparty.

2. Who the users are

NestUp is a **two-sided product**. The same person can be on both sides at different points in their life, and the same account supports both without switching modes.

Side A — the Seeker

Someone looking for a room in an existing shared apartment.

- Typically 22–35: students, young professionals, people relocating for work.
- Cares about: monthly rent, location, move-in date, and — the part other products ignore — who they will be living with.
- Their scarce resource is **time and travel**. Every apartment visit costs an evening.
- Their fear is committing to a year with people they cannot live with.

Side B — the Household (Lister)

An existing shared apartment with a room to fill.

- One member posts the room, but the room belongs to the household.
- Cares about: filling the room quickly, and filling it with someone who will not disrupt a flat that already works.
- Their scarce resource is **attention**. They cannot interview forty people.
- Their fear is a bad roommate they are then stuck with.

Who the customer is

For this project the **user and the customer are the same person**, and the product is free at the point of use. This is deliberate: a two-sided marketplace has no value to either side until it has liquidity on both, so charging early would suppress the very thing that makes it work.

The customer NestUp would be *built toward* is the **household with a vacancy**, because that is the side with urgency and a measurable cost of failure — an empty room is rent someone is paying out of pocket every month it stays empty. That is the side with willingness to pay, and the monetisation section below assumes it.

3. Business goals

GOAL	WHY IT MATTERS	HOW THE PRODUCT SERVES IT
Reduce wasted first meetings	The core value promise. If a seeker visits three apartments instead of ten and still finds a home, the product has paid for itself in evenings.	Compatibility scoring, and a swipe deck that only shows strong matches
Build two-sided liquidity	A matching product with rooms and no seekers, or seekers and no rooms, is worth nothing.	Public browsing with no signup wall; low-friction listing creation
Make the household the counterparty	Differentiator, and a genuinely better process.	Household-wide chat threads; co-posted listings; listing survives its poster
Keep members inside the product	If the conversation moves to WhatsApp on first contact, we lose the relationship and every signal with it.	In-app chat with realtime delivery, photo/video sharing, and viewing scheduling built into the thread
Earn trust	People are sharing where they live and who they are. One bad incident costs more than ten good matches earn.	Real authentication, per-row access control, reporting, blocking, suspension

How this would make money (not built — stated as intent)

The natural model is **paid promotion on the household side**: a listing that is not filling can pay to be placed higher in seekers' decks. It aligns with the value delivered — the product only charges when it is demonstrably doing the work — and it charges the side with urgency. A secondary line is a verification badge for both sides.

Nothing in the current product charges anyone. The schema and the ranking function would both need to change to support it, and that is noted honestly in the scale document as future work rather than implied to exist.

4. Software capabilities required

Working backwards from the goals above, these are the capabilities the product had to have.

4.1 Identity and trust

- Email + password authentication with **mandatory email confirmation** (a deliberate decision — it is the cheapest barrier against throwaway accounts in a product where people meet in real life).
- Password recovery.
- A profile that carries enough about a person to match on.
- Reporting, blocking, and account suspension.
- Account deletion that does not destroy other people's data.

4.2 Supply: rooms

- Create, edit, pause, mark-as-taken and remove a listing.
- Photos, with a check that a photo of a bedroom is actually a bedroom.
- A real geographic location.
- Multiple roommates able to co-own one listing.

4.3 Demand: discovery

- Public browsing, filterable and sortable, with no account required.
- A map of every placed room.
- A personalised swipe deck for signed-in members with a complete profile.

4.4 The matching itself

- A **Lifestyle** score: budget, city, move-in date, smoking, pets, cleanliness, sleep schedule, guests, noise, diet, Shabbat.
- A **Social** score: shared interests.
- A gate so the deck shows strong matches only, rather than everything sorted.

4.5 Communication

- One chat thread per (listing, seeker) pair, containing the **whole household**.
- Realtime delivery, read state, unread counts.
- Photo and video messages.
- Viewing proposal / approval inside the thread, with optional Google Calendar export.

5. Main user flows

5.1 Seeker: discover → match → meet

1. **Browse without an account.** `/browse` is public. Filter by city, rent, move-in date, lease length, roommate count, pets, smoking, balcony, A/C, parking, elevator, furnished, safe room. Sort by newest or price. Open the map to see every placed room at once.
2. **Open a room.** Photos, full details, who already lives there, and a map of the room and its neighbours.
3. **Sign up and build a profile.** Name, age, occupation, bio, photo; budget, preferred cities, move-in date, lease length; the Daily-life questionnaire (how *I* live) and roommate preferences (what I want in others); interests.
4. **Swipe.** `/swipe` shows a ranked deck of rooms scoring ≥ 60 combined. Like or skip. A like optionally opens a chat with a pre-filled hello.
5. **Chat with the household.** Every roommate is in the thread.
6. **Schedule a viewing** inside the chat. The household approves, and either side can export it to Google Calendar.

5.2 Household: list → screen → fill

1. **Create a listing.** Address (geocoded, or drag the pin), rent, rooms, size, available-from date, lease term, amenities, house rules, up to five photos.
2. **Tag the roommates** who live there. Each gets an invitation; accepting makes them a co-owner who can edit the listing and appears in the household.
3. **Receive interest** — likes and messages, all in one inbox.
4. **Screen with real information.** Every seeker who writes has a profile with a compatibility score against the room.
5. **Approve a viewing** from within the chat.
6. **Close the room** — mark taken, pause, or remove. Existing chats survive.

5.3 Cross-cutting flows

- **Account management:** notification and contact-visibility settings, blocked-member list, email/password change, account deletion (which hands a shared listing to a roommate rather than deleting it).

- **Moderation:** report a member or a listing; blocking is mutual and removes them from both decks; a suspension closes the app immediately.

6. What is deliberately not in scope

Stating these matters as much as the feature list — the assignment asks for clear thinking, not maximum surface area.

- **No payments.** No deposits, no rent collection, no escrow. Money moves outside the product.
- **No contracts or e-signature.** A lease is a legal instrument; faking one would be worse than omitting it.
- **No identity verification.** Email confirmation only. Real KYC needs a vendor and a budget.
- **No native mobile app.** The web app is mobile-first and installable as a PWA.
- **No recommendation learning.** Ranking is a deterministic scoring function, not a trained model. It is explainable, testable, and debuggable — which at this stage is worth more than being marginally better.

7. How success would be measured

Were this real, these are the numbers that would matter — chosen so that the product cannot look successful while failing its users.

METRIC	WHY THIS ONE
Viewings per successful match	The core promise. Lower is better.
Share of decks that are not empty	A personalised deck that shows nothing is the main failure mode of a strict match gate.
Time from listing published to room filled	The household's actual problem.
Reply rate to first message	Whether the two sides are genuinely well matched, or just matched.
Reports per thousand conversations	Trust. A rise here outweighs a rise in everything else.
